

Accelerating A* Pathfinding on Multi-Core Architectures through Parallelism

1st Fawaz Altahini

*College of Computer Science and Information Technology
Imam Abdulrahman Bin Faisal University
Dammam, Saudi Arabia
2220003471@iau.edu.sa*

2nd Ali Alhushayyish

*College of Computer Science and Information Technology
Imam Abdulrahman Bin Faisal University
Dammam, Saudi Arabia
2220005429@iau.edu.sa*

2rd Abdulrahman Alqahtani

*College of Computer Science and Information Technology
Imam Abdulrahman Bin Faisal University
Dammam, Saudi Arabia
2220004752@iau.edu.sa*

4th Ibrahim Alghamdi

*College of Computer Science and Information Technology
Imam Abdulrahman Bin Faisal University
Dammam, Saudi Arabia
2220006305@iau.edu.sa*

Abstract—A* pathfinding is a widely used algorithm in artificial intelligence applications such as robotics, navigation systems, and logistics due to its optimality and heuristic-guided search. However, its inherently sequential design limits performance when applied to large-scale or complex search spaces, resulting in high execution time and poor scalability. This project investigates and implements multiple parallelization strategies to accelerate A* search on multi-core CPU architectures using Python's multiprocessing framework. Three parallel approaches are explored: Batch Processing A*, Bidirectional A*, and Region-Based A*, each targeting different performance bottlenecks such as workload distribution, search depth, and memory scalability. Extensive benchmarking was conducted on grid sizes ranging from small to very large environments, measuring runtime, speedup, efficiency, and scalability across multiple CPU cores. Experimental results show that parallel implementations incur overhead for small problem sizes but significantly outperform the sequential baseline for large grids, where computational workload dominates communication and synchronization costs. Among the proposed strategies, Region-Based A* achieved the highest speedup and efficiency, demonstrating superior scalability on large search spaces. The study highlights that effective parallelization of A* requires careful consideration of synchronization, communication overhead, and workload decomposition, emphasizing that performance gains depend on both algorithmic design and system-level constraints.

Index Terms—Keywords: A* Pathfinding, Parallel Computing, Multi-Core Architectures, Python Multiprocessing, Heuristic Search, Performance Benchmarking, Scalability

I. INTRODUCTION

A popular and trusted method of is the A* algorithm. identifying the best solutions in different fields, such as robotics, logistics, and navigation. This is its effectiveness due to its ability. to take a combination of actual cost and heuristic estimations to de- find the shortest path effectively. However, the algorithm's sequential design is inherently limited in performance to large or search spaces that are extremely sophisticated. As the size of the problem with increases, the

computational load increases exponentially, causing big delays in real-time applications.

Robotic, modern applications, e.g. autonomous vehicles, swarms, and massive logistic systems, require speedy route. responsiveness and calculation. The sequential nature of A* makes its practical use in such cases impracticable, establishing a absolutely urgent need of optimization. Parallel computing on multi- A solution is offered by core architectures by distributing. calculating activities, lessening bottlenecks, and enhancing. overall throughput.

This is a project, "Accelerating A* Pathfinding on Multi-Core. Parallelism Architectures," is interested in improving the. scaling and performance of the A* algorithm. By leveraging Parallelization schemes such as batch processing, bidirectional search, and region-based decomposition. solution is in pursuit of accelerated pathfinding without sacrificing. optimality. The final objective is to provide a high-scale, high-rate, delivery. effective performance implementation that is able to exploit more core processors and are a convenient demonstration of parallel computing in pathfinding AI. applications.

A. Objectives

The primary objective of this project is to increase the performance. of the A* pathfinding algorithm with the capabilities. of the current multi-core processors. The specific objectives are as follows:

- 1) Run a parallel form of the A* algorithm that efficiently allocates the node exploration and assessment. tasks across multiple cores.
- 2) Enhance computational efficiency through lessening the execution time is large and complicated search. spaces.
- 3) Compare performance increases as a result of benchmarking and marking with the traditional sequential. A* implementation.

- 4) Show scalability of the proposed solution on various hardware setups and applications including robotics and logistics.
- 5) Report and certify findings to make them reproducible, and contribute to the future study of parallel computing and optimization techniques.

B. Justification

A^* algorithm is a reliable and famous approach, to pathfind and to traverse a graph. However, its sequential nature constrains performance, particularly in the case of big performance, scale problems. As the availability of multi-core increases. There is a definite opportunity to enhance the efficiency of A^* through the introduction of parallelism. This project aims to make use of parallel computing to expedite the monitoring of algorithm and minimize processing time. Achieving this would be of practical advantage in many aspects, and the number of them is not exhaustive, where robotics, real-time navigation and logistics planning are involved. The speed of decision-making is required. Moreover, the project is an excellent learning experience to put ideas of to practice, optimization of parallel architecture and performance in a real-world context. Utilization of cloud computing resources like AWS EC2 instances, provides with an equal testing environment, scope as well as making it cost-effective.

II. LITERATURE REVIEW

Parallelization A^* pathfinding algorithm is a significant challenge due to the sequential nature of priority queue and the required ordering to get the optimal node expansion. We studied several approaches to parallelize A^* algorithm, most studies addressed bottlenecks in design and proposed solutions through four main approaches:

A. Reducing Synchronization Overhead on Multi-Core CPUs

One of the major challenges highlighted in our reviewed work is synchronization bottleneck that arises when multiple threads share centralized data structures in parallel A^* . Global priority queue is what traditional implementations rely on, forcing threads to block while waiting for access, which limits parallel efficiency. To fix the issue, many studies propose decentralizing the search frontier. For example, Allowing each core to have its own priority queue minimizes locking and improves processor utilization, restoring in near linear speedup on grid based tasks [6]. Similar reasoning appears in parallel ripple search, by letting independent wavefronts progress and meet dynamically which reduces software level blocking [7]. Another models that reduces synchronization are hash distributed models, these models reduce it by assigning state ownership based on hashing so that communication occurs only when required [12]. Collectively, these approaches share the same goal which is reducing synchronization overhead which is important for exploiting multicore parallelism effectively. This suggests that synchronization minimization is not known as a programming optimization but it is a key architectural requirement for parallel A^* search.

B. Abstraction and Hierarchical Pathfinding

The primary trade-off for the massive speed increase achieved by Hierarchical Pathfinding (HPA) is a potential, yet usually minor, degree of path suboptimality. This occurs because the search is constrained by the predefined entrances (the border crossings) between clusters. A theoretically optimal path might cut straight through a cluster close to its border, but HPA's high-level search is forced to route the agent from one designated entrance node to the next, which may require a slight detour within the cluster to reach the specific entry/exit point. The degree of suboptimality is generally bounded and depends directly on the size of the clusters and the density of the entrances; larger clusters or fewer entrances lead to a faster search but increase the potential for the path to be longer than the absolute minimum, although research consistently shows that HPA often delivers paths that are typically within 1% to 5% of the truly shortest path, which is highly acceptable for most real-world applications like video game AI where speed is paramount.

C. GPU Acceleration and SIMD Architecture

Recent studies use GPUs and SIMD architectures to overcome the sequential nature of A^* pathfinding, enabling massive parallelism and higher throughput. However, there are key challenges in this architecture like overhead from communication, thread synchronization, and load balancing, which limits utilization of parallelism. To address these challenges, researchers have proposed many solutions. To optimize for SIMT execution, Zhou and Zeng proposed a GPU-targeted reworking of A^* (GA^*) which replaces the sequential ordering of node expansion with parallel bucketization exploiting GPU bandwidth [13]. Inam et al. improved GPU-based A^* efficiency by introducing hierarchical map abstractions, batched operations and GPU-optimized data structures to enable real-time multi-agent pathfinding, minimizing overhead from communication via per-kernel work aggregation and reducing thread synchronization costs by avoiding heavy atomic operations [14]. To address the warp divergence and load imbalance in IDA^* , S. Horie and A. Fukunaga proposed an approach called Block-Level IDA^* (BP- IDA^*) that parallelizes IDA^* by assigning separate subtrees to CUDA blocks with shared memories, synchronizing operations [3]. V. Rahmani and N. Pelechano addressed the challenge of efficiently computing paths for many agents in complex environments. They improved hierarchical A^* for navigation meshes (HNA^*) by precomputing connecting paths (PCCP) and exploiting GPU parallelism (CUDA HNA^*) to compute start-to-goal paths simultaneously, increasing lookup speed and enabling parallel pathfinding across all hierarchy configurations [4]. Hybrid CPU-GPU solutions were also proposed, by distributing depth-first search across multi-core CPUs to generate large state batches while the GPUs handle the heuristic evaluation, this approach balances workload and reduces overhead [2]. Finally, for large-scale workloads that exceed single device memory, He et al. proposed DA^* , which scales A^* across multiple

GPUs by partitioning the search space and overlapping computation with communication, reducing global synchronization [15].

D. Specialized Architecture and Multi-Agent Strategies

Multicore CPU based and GPU based architectures have been greatly used in supporting and improving multi-agent pathfinding algorithms. The Hierarchical Cooperative A* (HCA*) is used to coordinate the searches of the agents across different cores with the least amount of synchronization overhead. This parallel strategy enhances performance especially in heavily agentized environments; concurrent planning and conflict minimization is achieved using a global reservation system. Conflict-Based Search (CBS) considers another approach whereby the entire global search is divided into smaller A star searches of each agent. These personal searches are carried out simultaneously eliminating contradictions where there are conflicts. The technique allows simultaneous planning of many agents, and is therefore superior to both sequential CBS and simple many-Astar approaches. NUMA-Aware Parallel A In such multi-core systems, NUMA-Aware Parallel A optimizes access to memory through search space partitioning, which has a core accessing memory locally. This minimises memory contention, especially in problems with large state-space that can be difficult to solve using traditional methods. Finally, Parallel Anytime Heuristic Search (Weighted A) makes use of parallelism to search a bigger search space by allocating distinct cores to various frontier areas. It is fast and has a better quality of solution and is suitable in multi-agent systems that need solutions to be found simultaneously. The combination of parallelism and efficient memory management achieved by these strategies based on the multi-agent technique and special purpose hardware demonstrates the ways the scalability and performance of pathfinding algorithms can be greatly improved in complex, real-time applications.

III. METHODOLOGY

Parallelism was implemented as multiprocessing in Python library. Our selection of Python, as opposed to C/C++, can be explained by a number of reasons: First, A* is an artificial intelligence core algorithm. research and practice and Python most. applied in artificial intelligence field. Second, Python offers high-level expression of intricate data structures which A*. Python as well relies on like priority queues and dictionaries. is capable of parallel testing faster than can be done in C/C++ because. to their manual memory government that is hard and time consuming with the elaborate data structures. Lastly, Python Large grids that are handled by multiprocessing are serialised. applied in applications of A* In the real world. would be forced to install a low-level message mechanism. passing between processes.

According to our literature reviews A* pathfinding algorithm has many ways of parallelization, you can reduce synchronization overhead, use GPU, change the algorithm structure, and many other ways. In this project we selected 3 strategies for parallelizing A*:

A. Strategy 1: Batch Processing A*

Approach: Sequential A* suffers from resource underutilization when multiple agents need to compute their paths at the same time, for example if you had 50 agents the 50th agent needs to wait for the 49 agents ahead of him to finish causing throughput bottleneck. Batch processing solves this problem by treating each agent's path as independent for all others, so we distribute each agent request across all available CPU cores.

Changes to Code: We replaced the sequential A* loop that iterates over all pathfinding requests with a parallel version. Each request job contained the grid, start, and goal positions. All jobs were batch processed with Python's multiprocessing. Pool which distributes each job in the pool to run an independent sequential A* in a CPU core using pool.map() mechanism.

Parallel Computing Principles: Batch Processing is a type of data parallelism. The same algorithm is applied across different types of data leading to higher throughput than the sequential implementation.

B. Strategy 2: Bidirectional A*

Approach: This strategy addresses the problem with the exponential depth growth of search trees. Bidirectional A* uses 2 parallel workers 1 searches forward from the start to the goal position and the other searches backward from the goal to the start position. Both workers have a shared memory to detect when their searches intersect then compute the minimum path costs between all intersection points. This approach cuts the depth of the search by half reducing both latency and memory consumption.

Changes to Code: We implemented a parallel shared memory using multiprocessing.Array which enables lock-free communication between workers. The shared array contained the grid, cost arrays, and parent arrays this ensures that both workers can see each other's progress which helps detect if a node has already been visited by the other worker. To ensure that we have the optimal solution we used multiprocessing. Lock to update the shared variable best cost whenever a new intersection point is found ensuring race-free updates.

Parallel Computing Principles: Bidirectional A* uses shared-memory parallelism and asynchronous communication. We also reduce IPC overhead by using shared buffers instead. Bidirectional A* is also considered a type of task decomposition where 1 task is decomposed into 2 parallel sub-tasks.

In our parallel solution we implemented 3 strategies to parallelize A* search algorithm. We are going to solve this assignment using our 2^{nc} strategy: Bidirectional A*, which basically search a grid from the start (forward search) and the end (backward search) at the same time using 2 workers (processes) and a third process the coordinator to merge results.

1) **Race Condition Analysis:** There are 3 potential race condition variables:

1-best cost: Multiple processes can read and write to it at unpredictable times to terminate their search, and main process writes to it when it finds better path.

```
# Shared variable
best_cost = Value('d', float('inf'))

# Worker process
current_best = best_cost.value
if current_best < float('inf'):
    if current_f >= current_best:
        break
# Main Coordinator
new_meeting, new_cost = find_meeting_point(g_fwd, g_bwd,
    best_cost.value)
if new_meeting is not None and new_cost < best_cost.value:
    best_meeting = new_meeting
    best_cost.value = new_cost
```

Listing 1. Shared Best Cost Handling

2- stop_flag: Workers constantly read this shared flag to know when to stop, while the coordinator (main process) writes to it when a path is found or timeout occurs.

```
# Shared variable
stop_flag = Value(ctypes.c_bool, False) # Shared across
    processes

# Worker Process (READ)
while open_list and not stop_flag.value:
    # Process nodes
    pass # <-- Worker loop continues until stop_flag is
        set
# Main Coordinator (WRITE)
if elapsed > TIMEOUT:
    print(f"Timeout after {elapsed:.15f} s")
    stop_flag.value = True # Signal workers
        to stop
    print("Stopping search")
```

Listing 2. Race Condition: Stop Flag Handling

3-result_queue: Multiple workers (processes) can write to it at the same time.

```
# Shared object
result_queue = ctx.Queue() # Shared across processes

# Worker Processes (WRITE)
# Both forward and backward workers write to the same queue
result_queue.put((update, direction, middle_nodes,
    nodes_processed)) # <-- MARKED
# Main Coordinator (READ)
# Coordinator reads messages from the queue
msg = result_queue.get(timeout=0.2) # <-- MARKED
```

Listing 3. Race Condition: Result Queue Handling

C. Strategy 3: Region-Based A*

Approach: This strategy addresses the scalability issue of sequential pathfinding by dividing grids into smaller regions instead of handling the whole grid at once which might overburden the memory. Region-based A* implements a hierarchical solution with 2 levels. Low-level search runs a standard A* search within each region to find the optimal path between the entry and exit point of the region, this level of search can use multiple workers to run in parallel. High-level search predetermines the entry and exit waypoints of each region along the start to goal path then finds the optimal sequence of regions connecting the start and goal positions.

Changes to Code: We replaced the sequential A* full grid search with hierarchical region-based implementation. Our implementation began by the high-level search computing the waypoints along the diagonal line from start to goal positions and dividing the path into regions (splits). Then we used multiprocessing. Pool to distribute the low-level searches for each region across available CPU cores to run in parallel. After all parallel workers complete the low-level search, the high-level search combines the local paths to form the final path from start to goal.

Parallel Computing Principles: This strategy is an application of domain (spatial) decomposition where we split the large search grid into smaller independent sub-domains (regions) to run in parallel. Region-based is a hybrid of both data and task parallelism. Data parallelism in the low-level search where the same algorithm runs on different regions (data). Task parallelism in the high-level search where the pathfinding algorithm is split into multiple tasks like computing waypoints and connecting regions. Region-based A* reduces the computational load per worker using spatial decomposition, improves memory consumption with data parallelism, and provides high scalability with task parallelism.

IV. IMPLEMENTATION OF SYNCHRONIZATION

Run the code with various options (critical, atomic, reduction) of solving the race condition and see both the result and the execution time. We are using Python multiprocessing it doesn't have directives like in OpenMP like critical, atomic, etc., but you can replicate similar behavior:

1- Critical: in multiprocessing in python it has Value object which have internal locks used for variables like:

```
# Shared variables

# Best cost shared across processes
best_cost = Value('d', float('inf'))

# Stop flag shared across processes
stop_flag = Value(ctypes.c_bool, False)
```

Listing 4. Shared Variables Declaration

both here are multiprocessing. Value objects which their access is protected.

2- Atomic: we are using atomic operations with Value object whenever you use the object.value because of the internal lock the operation acts as atomic like in:

```
# Update shared best cost
best_cost.value = new_cost
```

Listing 5. Updating the Shared Best Cost

3- Python doesn't have reduction or a similar operation to it so we implemented a similar operation manually in:

```
# Find new meeting point
new_meeting, new_cost = find_meeting_point(g_fwd, g_bwd,
    best_cost.value)

if new_meeting is not None:
    if best_meeting is None:
        best_meeting = new_meeting
        best_cost.value = new_cost
```

Listing 6. Updating Best Meeting and Best Cost

What happens here we have 2 workers forward and backward each calculate their partial path cost then this function takes both calculations and find the minimum total cost and informs both workers.

V. EXPERIMENTAL RESULTS AND DISCUSSION

A. Sequential Performance Benchmark

In this section we analyzed the performance of the code in its sequential version before implementing parallelization.

TABLE I
BENCHMARK SUMMARY

Grid Size	Avg Time (s)	Success Rate
5x5	0.000017	100.0%
25x25	0.001559	100.0%
50x50	0.006276	100.0%
100x100	0.028403	100.0%
250x250	0.136912	100.0%
500x500	0.601152	100.0%
750x750	1.486550	100.0%
1000x1000	2.826670	100.0%
2500x2500	19.197305	100.0%

CUMULATIVE PERFORMANCE PROFILING RESULTS. Most Time-Consuming Functions: 121140429 function calls in 118.606 seconds. Ordered by: internal time.

B. Parallel Version Benchmarking

After implementing our 3 parallel strategies we benchmarked our code using 12 CPU cores, various grid sizes ranging from 50×50 to 5000×5000 , and 34% obstacles density per grid we ran 5 trails for each strategy with the various grid sizes:

TABLE II
BENCHMARK COMPARISON OF PARALLEL VS SEQUENTIAL STRATEGIES

Grid	Strategy	Time (s)	Speedup	Efficiency
50×50	Sequential	0.0035	1.00x	-
	Batch	0.042	0.08x	0.7%
	Region-Based	0.683	0.01x	0.0%
100×100	Sequential	0.0165	1.00x	-
	Batch	0.048	0.34x	2.9%
	Region-Based	0.718	0.02x	0.2%
500×500	Sequential	0.146	1.00x	-
	Batch	0.067	2.18x	18.1%
	Region-Based	0.793	0.18x	1.5%
1000×1000	Sequential	3.07	1.00x	-
	Batch	0.117	26.2x	218.2%
	Region-Based	1.001	3.07x	25.6%
5000×5000	Sequential	48.03	1.00x	-
	Batch	5.033	9.54x	79.5%
	Region-Based	4.397	10.93x	91.0%

Based on the above table we can see that for small grids sequential implementation is faster than all parallel strategies this is due to IPC overhead, process initialization latency, memory copying costs, and Amdahl's law. Starting from 1000x1000 grid which is equivalent to 1 million nodes we can see all of our parallel strategies are getting faster than the sequential implementation and this is because of the computational workload have outweighed the overheads of

IPC, process initialization and other bottlenecks. The main reason for the increased workload is that search trees grow exponentially and the parallel strategies are effectively distributing the workload across CPU cores. Our largest grid size was 5000x5000 equivalent to 25 million nodes during it our sequential implementation required on average 48 seconds to find the optimal path and in real-life applications this is a huge latency, our batch strategy got on average 5 seconds per path achieving 9.54x speedup and 79.50% efficiency, our bidirectional strategy wasn't much faster than our sequential implementation it only got 1.24x speedup and this is because bidirectional A^* is limited to 2 workers so it can not utilize all CPU cores, region-based A^* is our fastest strategy in large grids it was the fastest with 10.93x speedup and it utilized CPU cores the best with 91% efficiency.

C. Race Condition Benchmark Results

Run the code with the race conditions and see if you can catch the wrong results. Record the execution time. In the race condition version all shared variables are unsynchronized, therefore it didn't perform well in any path with grids bigger than 5×5 :

TABLE III
BENCHMARK SUMMARY (STRATEGY 2: BIDIRECTIONAL A^* - RACE CONDITION)

Grid Size	Avg Time (s)	Success Rate
5×5	0.000010	100.0%
10×10	0.817079	40.0%
25×25	1.249734	0.0%
50×50	1.435568	0.0%
100×100	1.497947	0.0%
250×250	1.667410	0.0%
500×500	2.760087	0.0%
750×750	5.412653	0.0%
1000×1000	10.481167	0.0%

D. Bidirectional Speedup Summary

Bidirectional A^* algorithm is limited to 2 workers (processes) it cannot work on more than 2 therefore this is a comparison between the sequential A^* (1 process) and bidirectional (2 processes):

TABLE IV
BIDIRECTIONAL SPEEDUP SUMMARY

Grid Size	Seq Time	Bi Time	Speedup	Efficiency
5×5	0.000004	0.000001	4.48x	223.96%
10×10	0.000067	0.824930	0.00x	0.00%
25×25	0.000537	1.226011	0.00x	0.02%
50×50	0.002432	1.404611	0.00x	0.09%
100×100	0.009913	1.420057	0.01x	0.35%
250×250	0.083435	1.690117	0.05x	2.47%
500×500	0.267020	3.018896	0.09x	4.42%
750×750	0.849042	6.638435	0.13x	6.39%
1000×1000	2.305058	11.947992	0.19x	9.65%

Based on results, we can see that bidirectional A^* performs worse than sequential A^* for small grid sizes, because

Python's Inter-Process Communication and process management overhead, but we can see bidirectional A^* in large grid sizes increase efficiency.

E. Comparative Analysis: Sequential vs. Parallel

Our experiments reveal a distinct crossover point in performance between sequential and parallel implementations.

1) *Small to Medium Grids* ($< 1000 \times 1000$): For smaller search spaces, the **Sequential** A^* algorithm outperforms all parallel strategies.

- **Reasoning:** The computational cost of exploring these grids is lower than the overhead introduced by Python's multiprocessing (process initialization, memory serialization, and IPC).
- **Observation:** Parallel strategies exhibited negative speedup (slower than 1.0x) in this range, validating Amdahl's Law regarding the fixed costs of parallelization.

2) *Large Grids* ($\geq 1000 \times 1000$): As the problem size increases, the computational workload outweighs the parallelization overhead, allowing parallel strategies to demonstrate their value.

- **Sequential:** Performance degrades exponentially. For a 5000×5000 grid, execution took ≈ 48 seconds.
- **Region-Based Parallelism:** The most effective strategy, achieving a **10.93x speedup**. By decomposing the domain, it maximizes CPU utilization (91% efficiency) and keeps memory consumption manageable per core.
- **Batch Processing:** Highly effective for throughput, achieving a **9.54x speedup**. It is ideal for scenarios with multiple agents but limited by the fact that individual paths are still computed sequentially.
- **Bidirectional Search:** The least effective parallel strategy, achieving only **1.24x speedup**. It is structurally limited to two workers (forward and backward), preventing it from scaling with additional CPU cores.

Summary: While sequential execution is preferred for simple navigation tasks, Region-Based parallelization is the superior choice for large-scale, complex environments.

F. Scalability Analysis

The scalability chart (Scalability Analysis: Execution Time vs Grid Size) illustrates the execution time of all strategies with increasing problem size this was tested on 12 CPU cores.

Note, the lower execution time the better. We can see our sequential implementation is the fastest when grid sizes are less than 1000×1000 but after its execution time increases exponentially to become the slowest in the largest grids. Batch processing performance shows high throughput while in its core it uses sequential A^* , using parallelism shows the actual difference between a single CPU core and multiple cores where it has a faster execution time starting from 250×250 grid. Bidirectional A^* at the low grid sizes its performing worse than sequential because of the IPC and shared memory but after sequential start getting worse around 1000×1000 grid bidirectional is constantly showing a slight better performance because it uses 2 workers instead of 1 like the sequential does.

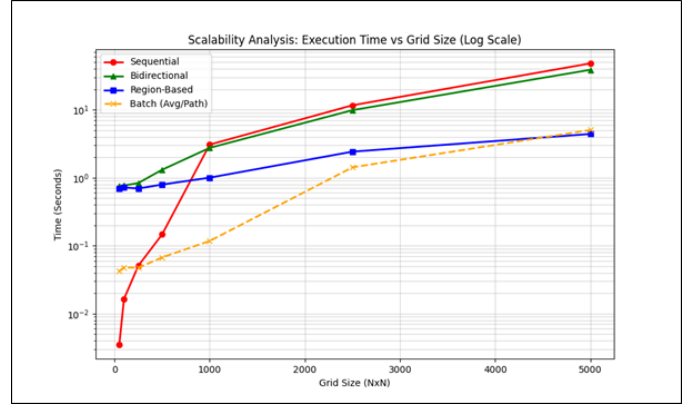


Fig. 1. Scalability Analysis: Execution Time vs Grid Size (Log Scale)

Region-based have the highest speedup and efficiency amongst all strategies causing it to have the best scalability as it shows a steady increase in execution time with increasing grids and this is because the problem size in region-based does not increase as much it increases in other strategies because it is splitting it into many regions.

The scalability chart (Strong Scaling: Execution Time vs CPU Cores) illustrates the execution time of our parallel strategies with different number of CPU cores on a 5000×5000 grid.

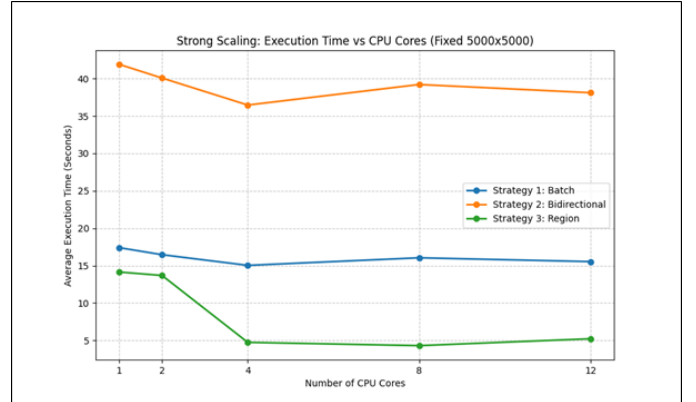


Fig. 2. Strong Scaling: Execution Time vs CPU Cores (Fixed 5000x5000)

Note, the lower execution time the better. Batch-processing performance is consistent ranging between 15 and 18 seconds, but it does not improve with increasing number of CPU cores, and this is due to several purposes like Python's multiprocessing manager fixed serialization overhead and the time spent communicating and managing process pool. Bidirectional A^* as we can see from the chart, has the highest execution time over any number of CPU cores compared to all parallel strategies, because bidirectional A^* in design cannot have more than 2 workers, but we can see increasing the number of CPU cores of Bidirectional A^* give us a slight improvement and this is because of the Operating System and Python's multiprocessing process management. Region-Based A^* has the highest scalability between all parallel strategies, it has

the fastest execution time with 8 cores where it found the optimal solution on a large grid in less than 5 seconds, this high scalability is due to domain decomposition where the large grids are divided into smaller ones running in parallel.

VI. LIMITATIONS

Parallel A^* is proposed to be implemented by the following ways. bound by the multiprocessing slowness of Python such as process creation, data serialization, which reduces the increase in performance with small and medium grid sizes. Bidirectional A^* can also only be scaled on two workers making. it can not be scaled to multi-core systems and Region-Based. In case of uneven obstacle in regions, A^* can be load-balanced. distributions. Competition with efficiency is brought by as well. the coordinating mechanisms that are used in race-avoidance. conditions. These limitations may be eliminated in future studies. by use of the algorithms in C++ using OpenMP. to reduce overhead, 1 to add dynamic load balancing to. a partitioning strategy, spreading parallelism to the GP-based implementation and strategy testing. stunning and authentic navigation environments.

VII. CONCLUSION

This project has been able to prove the effectiveness of leveraging of multiple core-based architectures to speed up the A^* pathfinding algorithm. Three of these need to be implemented and benchmarked by. different parallelization approaches- Batch Processing, Bidirectional Search, and Region-Based Decomposition—we iden- procedured definite performance lines between sequence and. parallel execution.

Consecutive sequential: preliminary experimental findings suggest that although the sequential method yields a better result than the parallel one, the opposite is not true. Sequential A sequential: preliminary experimental results suggest that the sequential method has a superior result compared to the parallel method, but the reverse is not the case.

It is in A^* implementation that is better with small grids. (1000 x 1000) because of the overhead of process initial- Parallel Python2.7, parallelizingationalization and inter-process communication. strategies open up to great benefits in complexity of problems. scales. Namely, the Region-Based A^* strategy was shown to be successful. be the most effective, with a speedup of 10.93x and an efficiency of 91.0% on 5000 x 5000 grids. Conversely, as Bidirectional A^* lowered the amount of search depth, it was not as scalable. was a two-worker design, which was in itself restrictive, and produced. only peripheral returns to the extremely scalable field. decomposition methods.

We also emphasized the extreme significance of synchronizing- tion mechanisms; management of race conditions failed. on large grids, where the success rate is 0 percent, and the reinforcement is required. because it is needed to manage a robust shared memory (locks and atomic). Parallel AI systems) operations.

Finally, as it is affirmed in this study, parallel computing is not. only an optimization but a need of real-time pathfind-

used in the large scale like autonomous robotics. and global logistics. Efforts in the future will lead to mitigation. Multi processing overhead in python with changing over to C++. using OpenMP and experimenting with massive parallelism using GPU. acceleration.

REFERENCES

- [1] H. Yang, S. Mukherjee, and M. Likhachev, "A-ePA*SE: Anytime Edge-Based Parallel A^* for Slow Evaluations," arXiv.org, 2023. <https://arxiv.org/abs/2305.04408>
- [2] "A Parallel CPU-GPU Framework for Batching Heuristic Operations in Depth-First Heuristic Search," Arxiv.org, 2021. <https://arxiv.org/html/2507.11916v2> (accessed Dec. 06, 2025).
- [3] S. Horie and A. Fukunaga, "Block-Parallel IDA* for GPUs (Extended Manuscript)," arXiv.org, 2017. <https://arxiv.org/abs/1705.02843> (accessed Dec. 07, 2025).
- [4] V. Rahmani and N. Pelechano, "Multi-agent parallel hierarchical path finding in navigation meshes (MA-HNA*)," Computers & Graphics, vol. 86, pp. 1-14, Feb. 2020, doi: <https://doi.org/10.1016/j.cag.2019.10.006>.
- [5] W. Du, F. Islam, and M. Likhachev, "Multi-Resolution A^* ," arXiv.org, 2020. <https://arxiv.org/abs/2004.06684> (accessed Dec. 07, 2025).
- [6] I. Jaffer, "Parallel A^* search on a multi-core CPU," Int. J. Adv. Res. Ideas Innov. Technol., vol. 6, no. 4, pp. 48-52, 2020.
- [7] A. Guzman and R. Bidarra, "Parallel ripple search: Scalable and efficient pathfinding for multi-core architectures," in Proc. Motion in Games (MIG), Edinburgh, U.K., Nov. 2011, pp. 226-237.
- [8] V. Vidal, L. Bordeaux, and Y. Hamadi, "Adaptive k-parallel best-first search: A simple but efficient algorithm for multi-core domain-independent planning," in Proc. Symp. Combinatorial Search (SoCS), Stone Mountain, GA, USA, Jul. 2010, pp. 127-128.
- [9] S. Sharma, S. Srijan, and J. V. Vidhya, "Parallelizing bidirectional A^* algorithm," in Advances in Parallel Computing, vol. 37, Proc. Int. Conf. on Comput. Intelligence and Data Eng. (ICCIDE), 2020, pp. 263-268.
- [10] A. Botea and M. Müller, "Near optimal hierarchical path-finding," J. Game Develop., vol. 1, no. 1, pp. 7-28, 2004.
- [11] S. Brand and R. Bidarra, "Parallel Ripple Search - Scalable and Efficient Pathfinding for Multi-core Architectures," 2011/2012
- [12] A. Kishimoto et al., "Evaluation of a Simple, Scalable, Parallel Best-First Search Strategy (HDA*)," arXiv/ICAPS evaluations (2012-2013).
- [13] Y. Zhou and J. Zeng, "Massively Parallel A^* Search on a GPU," AAAI/ACM (2015).
- [14] R. Inam, D. Cederman and P. Tsigas, " A^* Algorithm for Graphics Processors," 2010.
- [15] X. He, et al., "Efficient Parallel A^* Search on Multi-GPU System (DA*)," Comput. Sci. / journal (2021).
- [16] D. Silver, "Cooperative Pathfinding," AIIDE Conference, 2005.
- [17] R. Korf and M. Zhang, "Parallel Fringe Search for Shared Memory Multiprocessors," AAAI Workshop on Search Algorithms, 2000.
- [18] G. Sharon, R. Stern, and A. Felner, "Conflict-Based Search for Optimal Multi-Agent Pathfinding," AAAI Conference, 2012.
- [19] M. Likhachev et al., "Anytime Dynamic Planning and Parallel Weighted A^* ," Robotics Institute Technical Report, Carnegie Mellon Univ., 2009.
- [20] H. Zhou and P. Reinefeld, "NUMA-Aware Parallel A^* Search," Journal of Parallel & Distributed Computing, 2016.

APPENDIX A: SEQUENTIAL PARALLEL IMPLEMENTATION

<https://github.com/AbdulrahmanQht/ARTI503>